Machine Learning I

Tema 2. Aprendizaje Supervisado
2.1 Árboles de Decisión

Profesor: Raquel Martínez y Jose Tomás Palma

Curso 2024-2025

1 Introducción

2 ¿Qué es un árbol de decisión?

3 Construcción de árboles de decisión

4 Algoritmo básico

5 Espacio de hipótesis

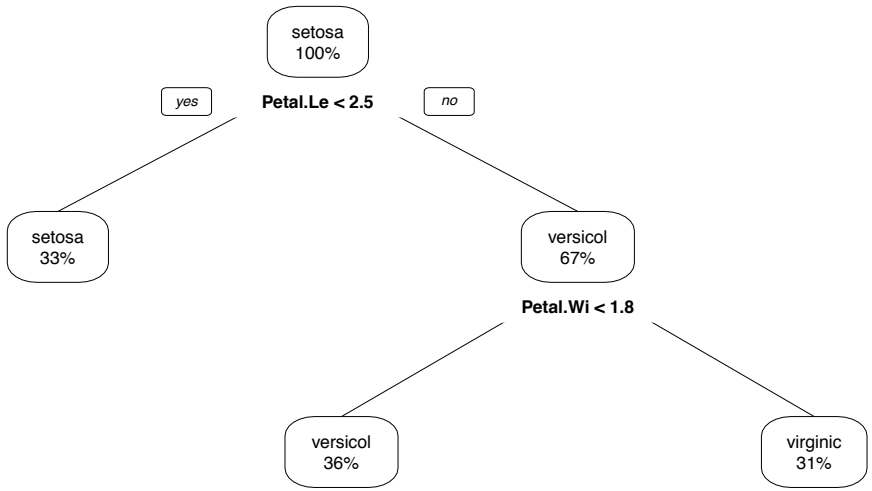
6 Sobre-aprendizaje

7 Consideraciones adicionales

8 Otros Algoritmos

Árboles de decisión

- Los métodos de clasificación basado en árboles de decisión son muy frecuentes en áreas como:
  - Diagnóstico, reconocimiento de patrones....
- Al ser fáciles de comprender se tiende a confiar más en ellos que en otros métodos (e.g. redes neuronales)



Árboles de decisión:

¿Cuándo utilizarlos?

- Cuando las instancias se pueden describir mediante un par <atributo,valor>.
- El conjunto de posibles valores de clase es discreto (aunque existen versiones de árboles de decisión para problemas de regresión).

Características:

- Las posibles soluciones son disyuntivas.
- Son tolerantes al ruido.
- Los datos de aprendizaje pueden contener errores.
- Los datos de aprendizaje pueden contener valores nulos en algún atributo para algún ejemplo.

Posibles aplicaciones:

- Tareas de diagnóstico.
- Análisis del riesgo crediticio.
- .....

Árboles de decisión

- Un árbol de decisión está formado por:
  - Un conjunto de nodos internos que representan atributos.
  - Un conjunto de arcos que representan posibles valores para los atributos (árboles binarios o multivaluados).
  - Nodos hojas, que representan el valor de clasificación
- Un árbol de decisión es capaz de representar cualquier subconjunto de instancias.
- En último termino se puede interpretar como una disyunción de reglas:

Si Pétalo.longitud $<$ 2.5 **then** iris setosa **O**
Si Pétalo.longitud $>$ 2.5 **y** Pétalo.anchura $<$1.8 **then** iris versicolor **O**
Si Pétalo.longitud $>$ 2.5 **y** Pétalo.anchura $>$1.8 **then** iris virgínica **O**

Árboles de decisión

- Un árbol de decisión también es una estructura recursiva que se puede definir como:

Definición recursiva

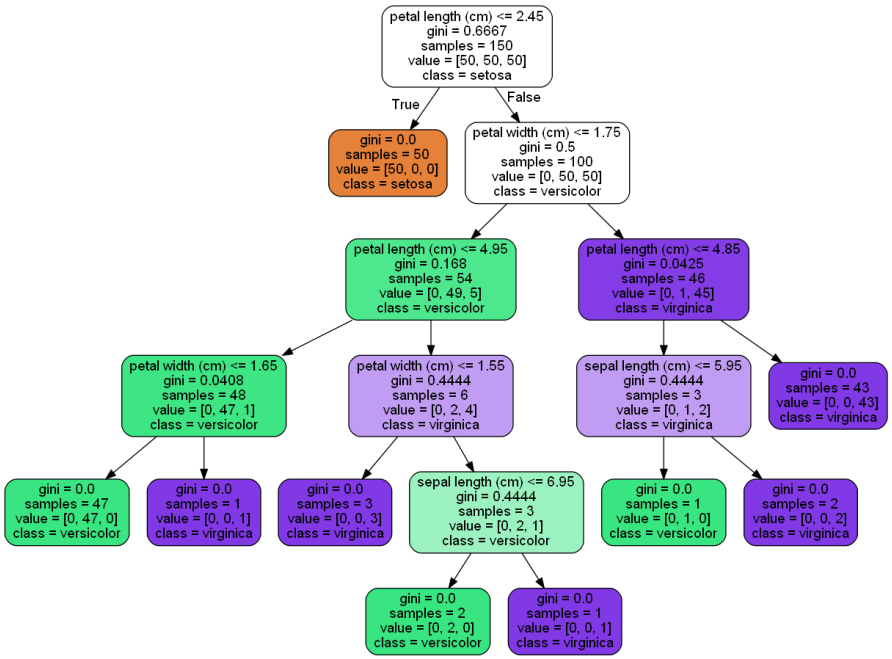
Un árbol de decisión T es una estructura formada por nodos, en el que existe:

- Un nodo raíz
- El nodo raíz tiene uno o más subnodos
- Cada uno de los subnodos puede ser, a su vez, raiz de un árbol

- Está característica recursiva hace que muchos de los algoritmos para crearlos se comporten también de manera recursiva.

Árboles de decisión

- La clave está en que cada nodo define una partición del conjunto de entrenamiento en función de los datos qué representa.
- Una vez elegido el criterio de partición, se puede volver a aplicar el mismo mecanismo para elegir los siguientes datos en cada una de las particiones.
- Claves:
  - ¿Qué particiones considerar?
  - ¿Qué partición considerar?



Construcción: Particiones posibles

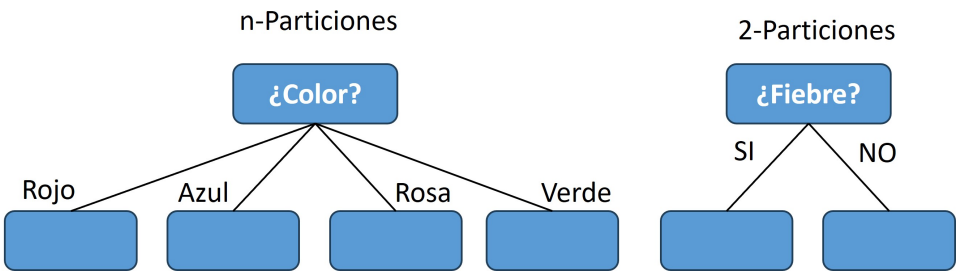
- Una partición es un conjunto de condiciones exhaustivas y excluyentes.
- Cuanto más tipos de condiciones permitamos:
  - Más posibilidad de encontrar patrones y más expresivos serán los árboles
  - Y probablemente obtengamos árboles más precisos.
  - Pero, a más particiones mayor complejidad: equilibrio entre complejidad y precisión
  - Esta es la razón por la que muchos algoritmos limitan el número de particiones.

Construcción: Particiones posibles I

- Los algoritmos más populares sólo proponen un tipo de partición para valores nominales y otro para valores numéricos:
  - **Particiones nominales**. En el caso que tengamos un atributo x_i que tenga como posibles valores $\{v_1, v_2, \cdots, v_n\}$, sólo es posible la partición

    $$(x_1 = v_1, x_2 = v_2, \cdots, x_n = v_n)$$

    que da lugar a árboles con nodos con más de dos nodos hijos.
    - En el caso de árboles binarios se tienen que evaluar n particiones (una por cada posible valor), definidas por $(x_i = v_i, x_i \neq v_i)$.

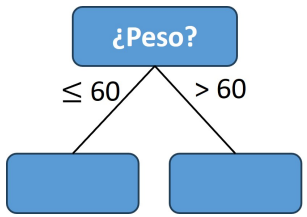


Construcción: Particiones posibles II

- **Particiones numéricas:** si el atributo x_i es numérico y continuo se intenta definir particiones que separe las instancias en intervalos de la forma

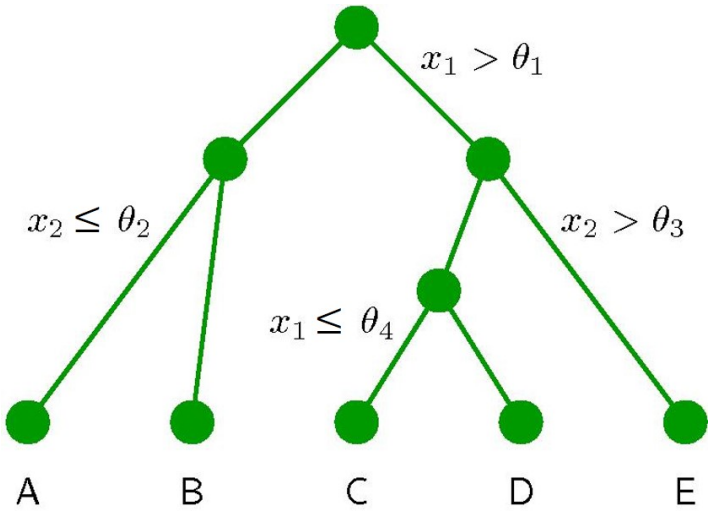
$$(x_i \leq a, x_i > a)$$

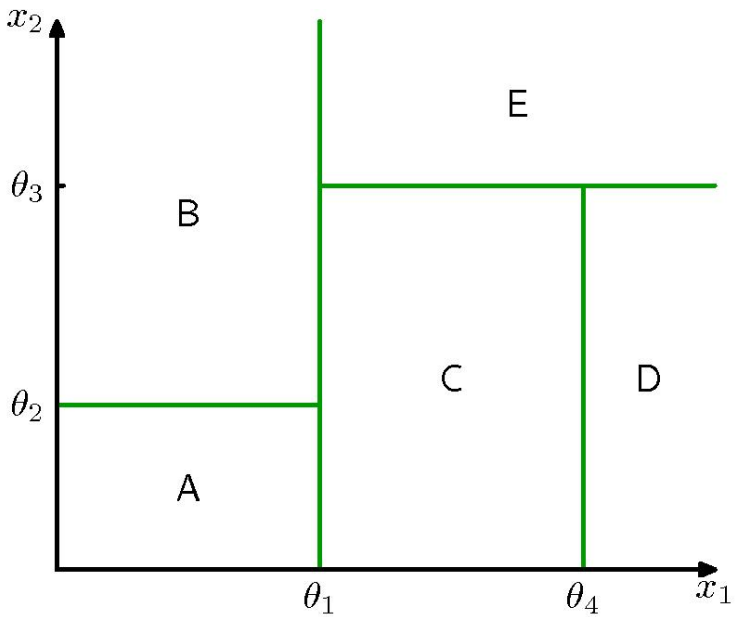
eligiendo diferentes valores de a tenemos diferentes particiones.



- La expresividad resultante se conoce como *expresividad proposicional cuadrangular* y que no relacionan atributos (sólo un atributo cada vez)

Expresividad cuadrangular





Construcción: Selección de particiones

- Incluso con la definición de particiones simples el número posible de particiones a analizar en cada caso pueden ser bastante grande.
- Como veremos, la mayoría de estos algoritmos son voraces, es decir, una vez que seleccionan una partición no la deshacen.
- Esto hace que el criterio para la selección de particiones sea clave para la eficacia del algoritmo.
- Sin embargo, el criterio elegido no debe ser muy costoso.

Algoritmo básico de aprendizaje

- El algoritmo básico de aprendizaje es el ID3 (Iterative Dichotomiser 3).

Algoritmo arbol $\leftarrow$ aprenderArbol($datos$)

1: **si** todos los ejemplos en $datos$ tienen la misma etiqueta **entonces**
2: **devolver** un nodo hoja con dicha etiqueta.
3: **sino**
4: Sea A el atributo que clasifica mejor a los objetos en $datos$.
5: **para todo** posible valor v de A **hacer**
6: $data(v) \leftarrow$ todos los objetos con $A = v$.
7: Añadir nueva rama $\leftarrow$ aprenderArbol($data(v)$).
8: **fin para**
9: **devolver** árbol.
10: **fin si**

Algoritmo ID3

ID3(Instancias, Etiquetas, Atributos)

Entrada: `Instancias`: el conjunto de datos.
Entrada: `Etiquetas`: el conjunto de posibles clases.
Entrada: `Atributos`: el conjunto de atributos en el conjunto `Instancias`.

si todos las instancias son positivos **entonces**
  devolver el nodo raíz con etiqueta $+$.
sino, **si** todos las instancias son negativos **entonces**
  devolver el nodo raíz con etiqueta -.
sino, **si** `Atributos`$=\emptyset$ **entonces**
  devolver el nodo raíz con el valor de `Etiquetas` más probable en `Instancias`.
fin si
Sea A el atributo que clasifica mejor a las instancias en $datos$.
Crear un árbol con un nodo etiquetado con A.
para todo posible valor v_i del atributo A **hacer**
  añadir un arco bajo la raíz con la comprobación $A = v_i$.
  sea `Instancias` $_{v_i}$ el subconjunto de `Instancias` con $A = v_i$.
  si `Instancias`$_{v_i} = \emptyset$ **entonces**
    añadir un nodo hoja al arco añadido con el valor de `Etiquetas` más probable en `Ejemplos`.
  sino
    añadir al nuevo arco el subárbol generado por **ID3(**`Instancias`$_{v_i}$**,** `Etiquetas`**,**`Atributos-`$\{A\}$**)**.
  fin si
fin para
devolver nodo raíz.

ID3: Selección de particiones

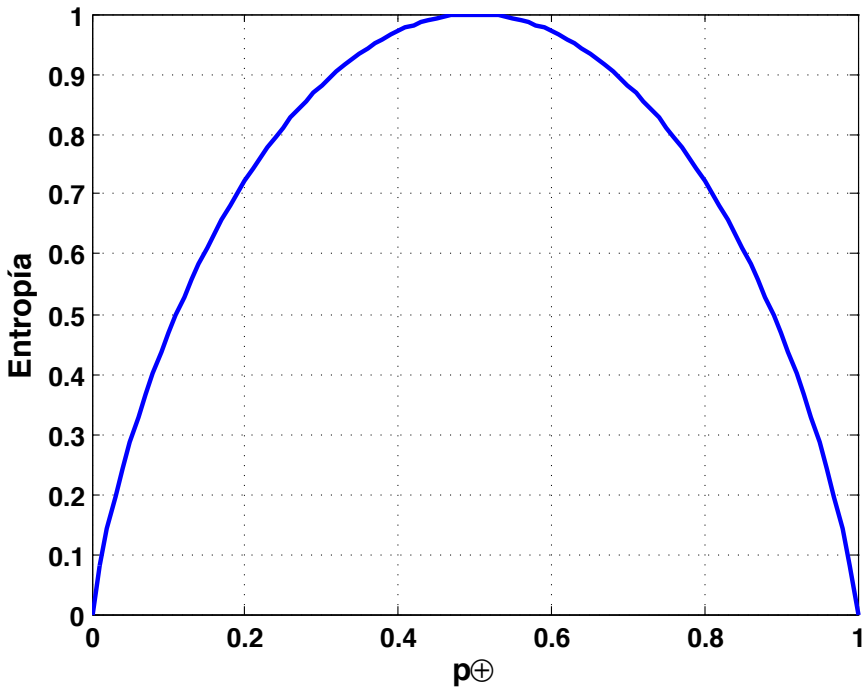
- La elección del mejor atributo para disgregar los nodos del árbol es clave para la eficiencia del clasificador resultante.
- Para determinar dicho atributo se utiliza la **entropía**

Sea S un conjunto de train. Sea $p_\oplus$ la proporción de instancias positivas en S. Sea $p_\ominus$ la proporción de instancias negativas en S. La entropía de S es:

$$H(S) = -p_\oplus \log_2 p_\oplus - p_\ominus \log_2 p_\ominus$$

ID3: Selección de particiones

- La entropía nos mide la homogeneidad de los datos
  - Cuanto más homogéneos (distribuciones idénticas) la entropía es igual a 1.
  - En ausencia total de homogeneidad (sólo hay presencia de instancias de una clase), la entropía es 0



ID3: Ejemplo

Día	Cielo	Temperatura	Humedad	Viento	Jugar
D1	Soleado	Calor	Alta	Flojo	No
D2	Soleado	Calor	Alta	Fuerte	No
D3	Nublado	Calor	Alta	Flojo	Si
D4	Lluvia	Templado	Alta	Flojo	Si
D5	Lluvia	Frio	Normal	Flojo	Si
D6	Lluvia	Frio	Normal	Fuerte	No
D7	Nublado	Frio	Normal	Fuerte	Si
D8	Soleado	Templado	Alta	Flojo	No
D9	Soleado	Frio	Normal	Flojo	Si
D10	Lluvia	Templado	Normal	Flojo	Si
D11	Soleado	Templado	Normal	Fuerte	Si
D12	Nublado	Templado	Alta	Fuerte	Si
D13	Nublado	Calor	Normal	Flojo	Si
D14	Lluvia	Templado	Alta	Fuerte	No

ID3: Selección de particiones

- En el ejemplo del tenis tenemos 9 objetos clasificados como $\oplus$ y 5 como $\ominus$, con lo que
  - $H([9\oplus, 5\ominus]) = -0.642 \cdot \log_2 0.642 - 0.358 \cdot \log_2 0.358 = 0.94$
  - $H([m\oplus, m\ominus]) = 1$ (datos totalmente homogéneos).
  - $H([0\oplus, 1\ominus]) = H([1\oplus, 0\ominus]) = 0$ (ausencia total de homogeneidad).
- Por lo tanto, a la hora de construir un árbol es preferible crear nodos con menos entropía (árboles pequeños).

ID3: Selección de particiones

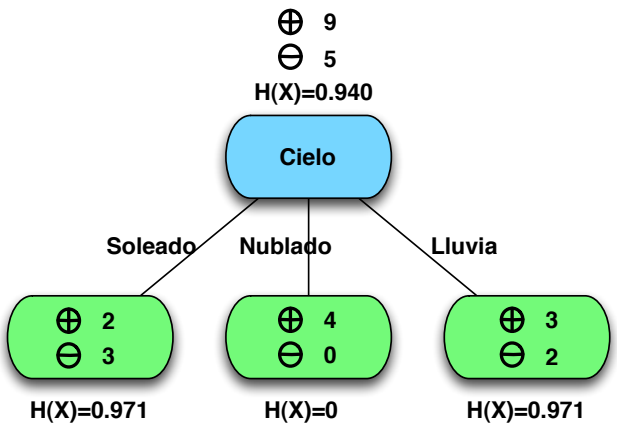
- Sea un conjunto de datos $\mathcal{X}$ con entropía $H(\mathcal{X})$.
- Si elegimos un atributo A para crear un nodo del árbol, la entropía esperada es:

$$H(\mathcal{X}, A) = \sum_{v \in valores(A)} \frac{|\mathcal{X}_v|}{|\mathcal{X}|} H(\mathcal{X}_v)$$

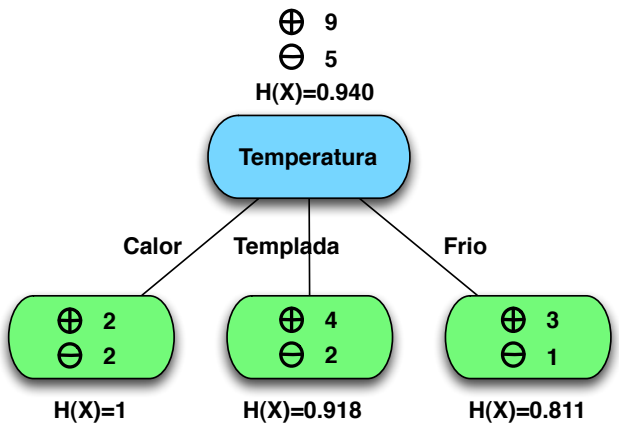
- Siendo $\mathcal{X}_v$ el subconjunto de $\mathcal{X}$ con todos las instancias con $A = v$.
- Por lo tanto, la reducción esperada de la entropía, o lo que es lo mismo la **ganancia de información**, al elegir el atributo A como nodo de decisión del árbol es.

$$Ganancia(\mathcal{X}, A) = H(\mathcal{X}) - H(\mathcal{X}, A) = H(\mathcal{X}) - \sum_{v \in valores(A)} \frac{|\mathcal{X}_v|}{|\mathcal{X}|} H(\mathcal{X}_v)$$

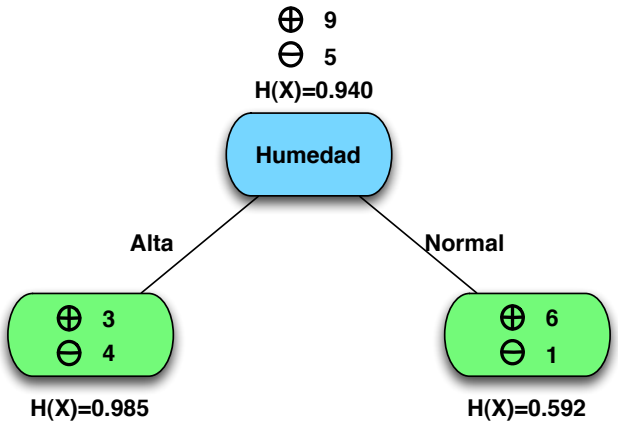
ID3: Ejemplo



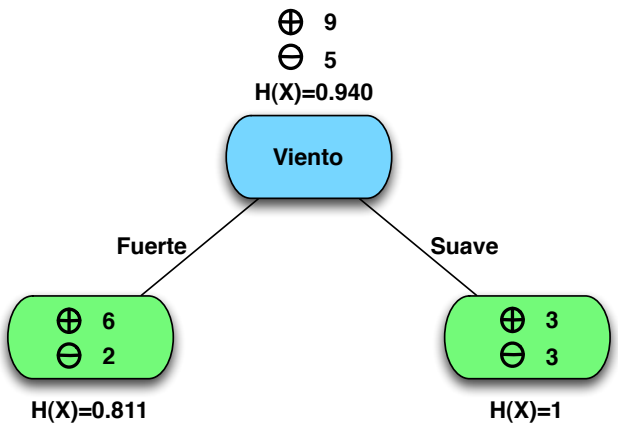
Ganancia(X,Cielo)=
0.94-(5/14) * 0.971- (4/14) * 0 -(5/14) * 0.971 = 0.246



Ganancia(X,Temperatura)=
0.94-(4/14) * 1 - (6/14) * 0.918 -(4/14) * 0.811 = 0.02

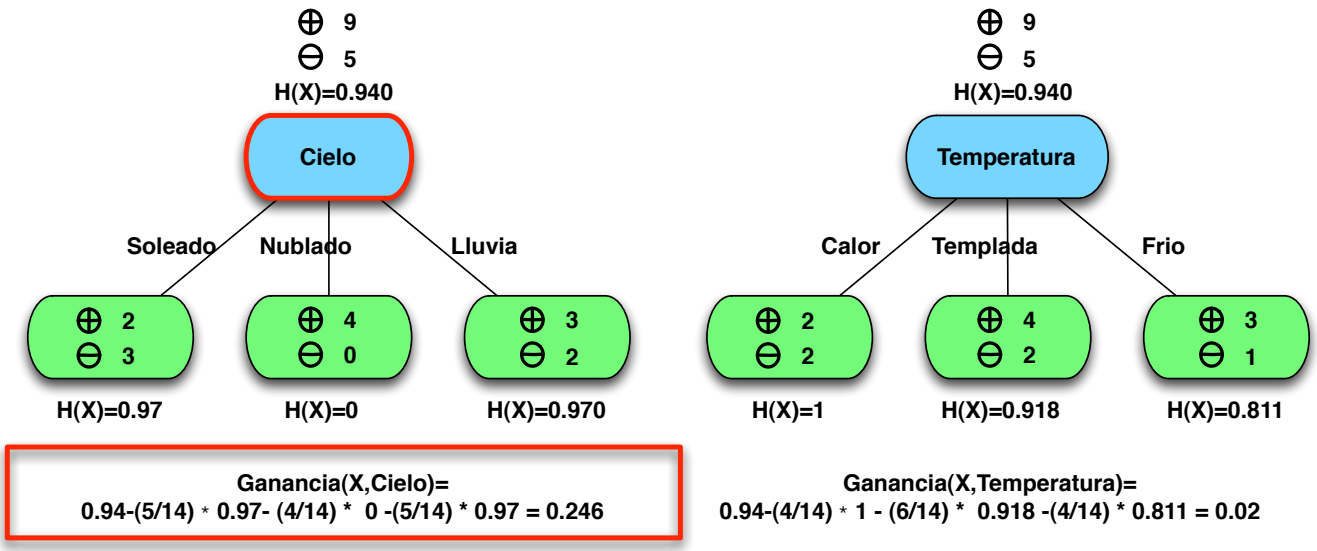


Ganancia(X,Humedad)=
0.94-(7/14) * 0.985 - (7/14) * 0.592 = 0.151



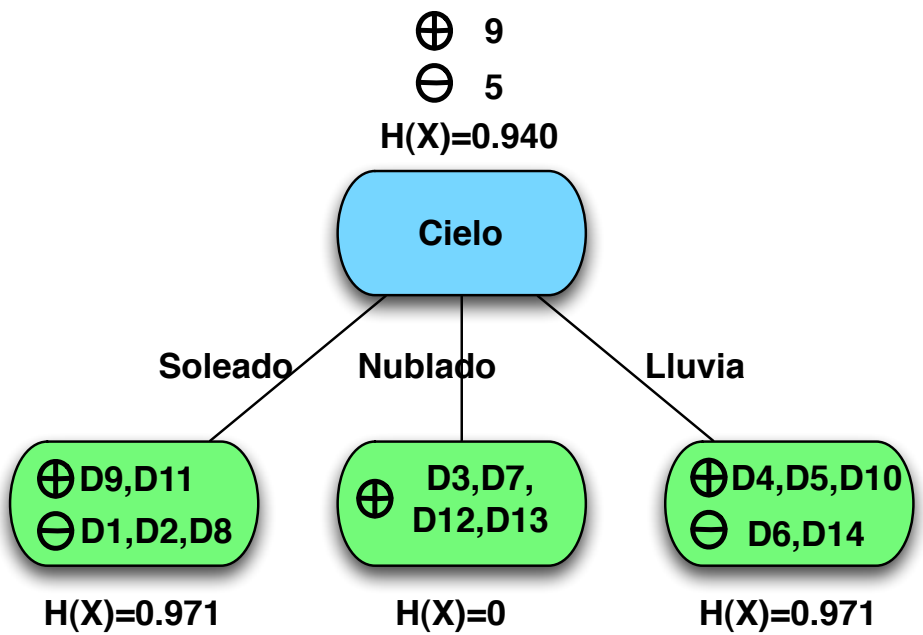
Ganancia(X,Viento)=
0.94-(8/14) * 0.811 - (6/14) * 1 = 0.048

ID3: Ejemplo


⊕ 9
⊖ 5
H(X)=0.940
Cielo
Soleado
Nublado
Lluvia
⊕ 2
⊖ 3
H(X)=0.97
⊕ 4
⊖ 0
H(X)=0
⊕ 3
⊖ 2
H(X)=0.970
Ganancia(X,Cielo)=
0.94-(5/14) * 0.97- (4/14) * 0 -(5/14) * 0.97 = 0.246
⊕ 9
⊖ 5
H(X)=0.940
Temperatura
Calor
Templada
Frio
⊕ 2
⊖ 2
H(X)=1
⊕ 4
⊖ 2
H(X)=0.918
⊕ 3
⊖ 1
H(X)=0.811
Ganancia(X,Temperatura)=
0.94-(4/14) * 1 - (6/14) * 0.918 -(4/14) * 0.811 = 0.02
⊕ 9
⊖ 5
H(X)=0.940
Humedad
Alta
Normal
⊕ 3
⊖ 4
H(X)=0.985
⊕ 6
⊖ 1
H(X)=0.592
Ganancia(X,Humedad)=
0.94-(7/14) * 0.985 - (7/14) * 0.592 = 0.151
⊕ 9
⊖ 5
H(X)=0.940
Viento
Fuerte
Suave
⊕ 6
⊖ 2
H(X)=0.811
⊕ 3
⊖ 3
H(X)=1
Ganancia(X,Viento)=
0.94-(8/14) * 0.811 - (6/14) * 1 = 0.048

ID3: Ejemplo

- Por lo tanto, el atributo que ofrece una mayor ganancia de información es el atributo **Cielo**.
- Utilizando **Cielo** como nodo raíz el árbol inicial quedaría:



ID3: Ejemplo

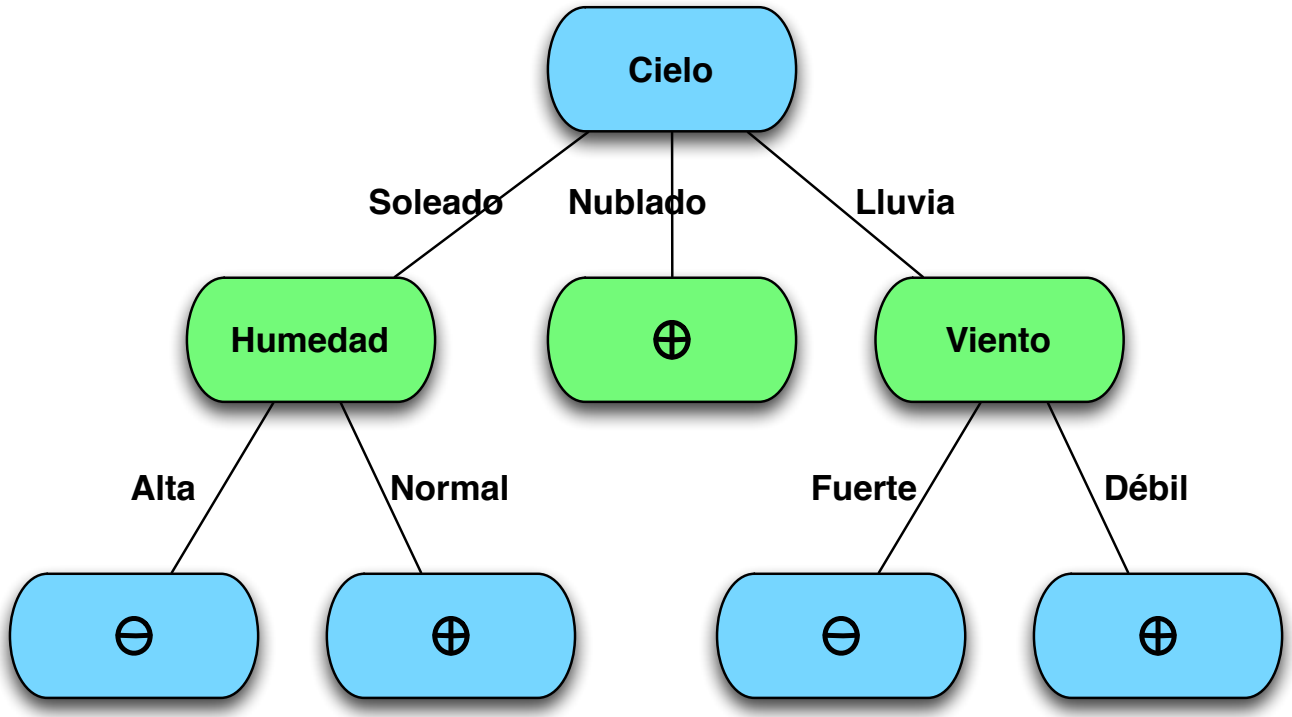
- Ahora habría que repetir el proceso con los nodos correspondientes a los valores **soleado** y **lluvia** (el nodo **nublado** sólo contiene una clase).
- Para el nodo $Cielo = Soleado$:
  - $\mathcal{X}_{soleado} = \{D_1, D_2, D_8, D_9, D_{11}\}$ con $H(\mathcal{X}_{soleado}) = 0.971$.
  - $Ganancia(\mathcal{X}_{soleado}, Temperatura) = 0.971 - \frac{2}{5} \cdot 0 - \frac{2}{5} \cdot 1 - \frac{1}{5} \cdot 0 = 0.570$
  - $Ganancia(\mathcal{X}_{soleado}, Humedad) = 0.971 - \frac{3}{5} \cdot 0 - \frac{2}{5} \cdot 0 = 0.971$
  - $Ganancia(\mathcal{X}_{soleado}, viento) = 0.971 - \frac{2}{5} \cdot 1 - \frac{3}{5} \cdot 0.918 = 0.019$

ID3: Ejemplo

- Para el nodo $Cielo = Lluvia$:
  - $\mathcal{X}_{lluvia} = \{D_4, D_5, D_6, D_{10}, D_{14}\}$ con $H(\mathcal{X}_{lluvia}) = 0.971$.
  - $Ganancia(\mathcal{X}_{soleado}, Temperatura) = 0.971 - \frac{3}{5} \cdot 0.918 - \frac{2}{5} \cdot 1 = 0.820$
  - $Ganancia(\mathcal{X}_{soleado}, Humedad) = 0.971 - \frac{2}{5} \cdot 1 - \frac{3}{5} \cdot 0.918 = 0.820$
  - $Ganancia(\mathcal{X}_{soleado}, Viento) = 0.971 - \frac{3}{5} \cdot 0 - \frac{2}{5} \cdot 0 = 0.971$

ID3: Ejemplo

- Por lo tanto, el árbol resultante sería



Espacio de hipótesis

- El espacio de hipótesis es completo (el árbol que buscamos está allí).
- El tipo de búsqueda es *top-down* con estrategia *hill-climbing* (*greedy*) por lo que no hay backtracking (mínimos locales).
- Es robusto respecto al ruido al utilizar una búsqueda estadística.
- Se tiene preferencia por árboles pequeños con los atributos con mayor ganancia de información cerca de la raíz.
- El modelo obtenido es completo y consistente.

La navaja de Occam

- ¿Por qué se prefieren las hipótesis más cortas?
- A favor:
  - El número de hipótesis pequeñas es menor que el de las más grandes.
  - Una hipótesis pequeña que se ajuste a los datos es menos probable que se deba a coincidencia.
- En contra:
  - Existen muchas formas para definir un conjunto pequeño de hipótesis

Sobre-aprendizaje

- Los árboles de decisión puede adolecer de *overfitting*.
  - El conjunto de hipótesis no es lo suficientemente representativo.
  - Los ejemplos tienen errores.

Sobre-aprendizaje

Dado un espacio de hipótesis H, se dice que una hipótesis particular $h \in H$ sobreajusta los datos de entrenamiento si existe una hipótesis alternativa $h' \in H$, tal que h presenta un error menor que h' sobre los ejemplos de entrenamiento, pero h' presenta un error menor que h sobre el conjunto total de observaciones.

Proceso de poda I

¿Cómo podemos evitar el sobre-aprendizaje?

- La idea es modificar el algoritmo para encontrar modelos más generales.
  - Eliminar condiciones de las ramas del árbol $\rightarrow$ Poda
- Existen dos tipos de poda
  - **Prepoda:** El proceso se realiza durante la construcción del árbol, estableciendo un criterio de parada para dejar de especializar nodos
    - El número de instancias por nodo
    - El error esperado
    - Minimun Description Lenght.

Proceso de poda II

- **Postpoda:** El proceso se realiza después de la construcción del árbol.
  - Se tiene una visión global del problema.
  - Consiste en ir el eliminado nodos de abajo a arriba mientras se vaya cumpliendo un criterio determinado.
- Se puden combinar ambas aproximaciones.

Consideraciones adicionales

- **Medidas alternativas**: En algunos casos se suele utilizar el $Ratio$ de la ganancia de información, para evitar el hecho de que se favorezca la selección de los atributos con más valores:

$$Ganacia_r(\mathcal{X}, A) = \frac{Ganancia(\mathcal{X}, A)}{H(\mathcal{X}, A)}$$

- Existen muchos criterios que se pueden utilizar GINI, Error esperado, DKM, basados en AUC, MDL,...

Consideraciones adicionales

- **Valores desconocidos**: Se puede intentar asignarle un valor.
  - El más probable
  - El más probable dentro de su clase.
  - Asignarles las probabilidades.
- **Atributos con coste**: Podemos dar más importancia a unos atributos que a otros:

$$\frac{Ganancia(\mathcal{X}, A)^2}{Coste(A)} \quad o \quad \frac{2^{Ganancia(\mathcal{X},A))} - 1}{(Coste(A) + 1)^w}, \; con \;\; w \in [0, 1]$$

C4.5, C5.0

- **C4.5**: Permite, a diferencia que ID3, que las características puedan ser continuas, definiendo de forma dinámica un atributo discreto particionando los atributos continuos en un conjunto discreto de intervalos.
  - C4.5 transforma los árboles obtenidos en un conjunto de reglas del tipo *if-then*. La precisión de cada regla es evaluada de forma independiente para determinar el orden en el que deben ser aplicadas.
  - Un proceso de poda elimina antecedentes de las reglas si con esto se mejora la precisión de la misma.
  - C5.0, utiliza menos memoria y obtiene un conjunto de reglas menor y más preciso.
  - J48 es la implementación en código abierto de C4.5

CART

- **CART** (Classification and Regression Trees): Muy similar al anterior, pero:
  - Permite que la variable que define la clase sea continua y no construye un conjunto de reglas.
  - Utiliza el índice de Gini en vez de la ganancia de información para seleccionar la mejor partición.
  - Utiliza también el esquema de partición recursiva utilizando una estrategia voraz.
  - También permite resolver problemas de regresión.

CART

- Se elige la partición que produce el menor valor de la función de coste.
  - Para regresión: RSME.
  - Para clasificación: GINI

$$Gini(p) = 1 - \sum_{i=1}^{n} (p_i)^2$$

con p_i las proporciones de instancias de la clase i en la partición

- Prepoda: Utiliza como criterio de parada el número mínimo de instancias asignadas al nodo.
- Postpoda: utiliza un criterio que controla la importancia relativa del error frente la complejidad (tamaño del árbol).

Random Forest

- Random Forest es una técnica que construye un gran número de árboles de decisión no correlacionados.
- Se basa en la técnica de Agregación de Bootstrap (Bagging), tecnica de agregación de clasificadores o regresores que:
  - Aumenta la precisión y estabilidad reduciendo los efectos del ruido.
  - Reduce la varianza en las predicciones
  - Ayuda a evitar el sobre ajuste.
- La predicción del modelo se elige analizando las predicciones de cada uno de los árboles considerados en el modelo.

Random Forest: Algoritmo

Algoritmo RF $\leftarrow$ RandomForest($datos$)

1: **para** $i \leftarrow 1$ to $n_arboles$ **hacer**
2: Extraer una muestra de tamaño $size(data)$ de $datos$ por bootstrapping
3: Construir un árbol T_i repitiendo recursivamente para cada nodo hoja
4: 1. Seleccionar aleatoriamente m atributos.
5: 2. Seleccionar la mejor partición de las inducidas por los m atributos.
6: 3. Dividir el nodo en dos nodos hijos.
7: 4. Si el tamaño del nodo alcanza n_{min} no continuar dividiendo.
8: **fin para**
9: **devolver** El conjunto de árboles $\{T_i\}_1^{n_arboles}$.

Random Forests: Algoritmo

- Si tenemos p atributos las recomendaciones para el valor de m son:
  - Clasificación: $\lfloor \sqrt{m} \rfloor$
  - Regresión: $\lfloor p/3 \rfloor$
  - Siendo el valor mínimo 1.
- Una vez se han construido los árboles para hacer predicciones:
  - Clasificación: La clase más votada por los árboles.
  - Regresión: La media de todas las predicciones.

Random Forests: OOB e importancia de las variables.

- **OOB (out of the bag) error estimate**: para cada muestra se predice el error utilizando sólo los árboles en los que no ha sido utilizada (no ha sido elegida en el bootstrapping)
  - Los valores son parecidos a los que se obtienen mediante una validación cruzada de N-pliegues.
- **Importancia de los atributos**:
  - En cada partición del árbol se registra la mejora del criterio de división.
  - Este valor se considera la importancia del atributo
  - Para cada variable se agregan los valores generados en cada árbol.

Conclusiones

- La utilización de árboles de decisión es muy popular en muchas disciplinas.
- Experimentalmente han demostrado una buena capacidad de clasificación.
- La clave reside en la función a optimizar a la hora de elegir el atributo para disgregar el árbol.
- Los ensambles nos permiten mejorar los resultados combinando información.

Bibliografía I

- José Hernández Orallo, M.José Ramírez Quintana, Cèsar Ferri Ramírez, 2004. Introducción a la Minería de Datos. Pearson education. Editorial Pearson, 2004. ISBN: 84 205 4091 9
- Jiawei Han, Micheline Kamber. *Data Mining: Concepts and Techniques*. Morgan Kaufman Publishers, 2003.
- Christopher Bishop. *Pattern Recognition and Machine Learning*. Springer-Verlag, 2006
- L. A. Breslow and D. W. Aha. *Simplifying decision trees: a survey*. Knowledge Engineering Review, 12(1):1–40, 1997.

Bibliografía II

- Sheerama K. Murthy. *Automatic construction of decision trees from data: A multi-disciplinary survey*. Data Mining and Knowledge Discovery, 1997.
- Ethem Alpaydin. *Introduction to Machine Learning*. MIT Press 2004.
- José T. Palma y Roque Marín (eds.). *Inteligencia Artificial: Métodos, Técnicas y Aplicaciones*. McGraw-Hill, 2008